Performance Analysis of Planar-Float and Direct Packed-BMP AVX2 Implementations for Image Grayscale Conversion

CEPARCO — Integrating Project

Mid-Project Milestone Report

Abdulrahman, Lim, Lopez, Nicolas

I. Summary

This report documents progress on the approved study of grayscale conversion using 24-bit BMP inputs. It examines the trade-off between separate single-precision color planes and direct processing of packed BGR pixels with x86-64 AVX2. The primary metric is complete in-memory conversion from the original packed input to the final grayscale BMP buffer.

The reorganized Windows x64 implementation compiled successfully and passed all five automated test suites. Sequential C, planar C, and planar NASM AVX2 also produced byte-identical 640 × 480 BMP output files in separate command-line comparisons. The direct packed-BGR AVX2 routine has passed extraction and guarded-boundary tests, but does not yet perform full grayscale conversion.

II. Week 10 Milestone Status

Proposal requirement	Observed progress
24-bit BMP reading/writing and sequential grayscale	Implemented; 14 BMP functional cases passed
Different widths, row padding, signed BMP height	Exercised by the functional and extraction tests
Initial planar-float NASM AVX2	Prototype assembled and linked on Windows x64; 90 AVX2 comparison cases passed
Preliminary direct packed-BMP SIMD	Eight-pixel BGR extraction validated: 78 cases and guarded-boundary test passed
Mid-project progress report and demonstration	Revised report draft prepared; group verification and live demonstration remain

The sequential and planar conversion paths have passed the available functional tests. The direct packed-BMP work remains an extraction-only prototype, so an application-level comparison between the two proposed AVX2 grayscale representations is not yet possible.

III. Implemented methods and Numerical Rules

Mode	Input and processing	Current status
Sequential C	Read packed BGR bytes, apply weighted grayscale, write 24-bit grayscale BGR	Functional reference
Planar C	Extract three float planes, compute grayscale in C, assemble output	Tested comparison mode
Planar NASM AVX2	Extract float planes, process eight values per YMM vector, handle tails in C	Working instructional prototype
Direct packed-BMP AVX2	Load exactly 24 packed BGR bytes for eight pixels and extract float channels with SIMD	Extraction prototype only

Each BGR input byte is converted to a 32-bit float. The intended evaluation is tR = 0.299f × R, tG = 0.587f × G, and tB = 0.114f × B, followed by Y = (tR + tG) + tB. The value is clamped to [0, 255], rounded to the nearest integer with ties-to-even, and written into each of the B, G, and R output channels.

The tested planar outputs matched sequential C byte-for-byte. A final code-level audit of instruction order, floating-point contraction, and conversion behavior is still needed before claiming equivalence for every possible valid input.

IV. Correctness and Memory-boundary Evidence

Windows test suite	**Result**
BMP reader/writer and sequential conversion	PASS - 14 cases (latest reorganized Windows build)
Planar C output vs. sequential C	PASS - 44 cases
Planar NASM AVX2 vs. sequential C	PASS - 90 cases
Benchmark CLI, output equality, and memory reporting	PASS - benchmark script
Packed BGR SIMD extraction vs. scalar C	PASS - 78 cases
Protected-page packed-load boundary test	PASS - guarded boundary check

Eight 24-bit pixels occupy exactly 24 bytes. The extraction prototype uses a 16-byte load and a separate 8-byte load, avoiding an unconditional 32-byte read at the end of a pixel group. Its C driver checks row boundaries and handles fewer than eight remaining pixels separately.

The current Windows test run passed 226 correctness cases plus the benchmark-framework and protected-boundary tests. Separate binary comparisons found no differences between sequential C, planar C, and planar AVX2 BMP outputs for a generated 640 × 480 test image. These checks support correctness for the tested inputs; they do not establish universal memory safety or numerical equivalence.

Visual Verification of Grayscale Conversion



From L-R: Original BMP, Sequential C, Planar AVX2

To demonstrate the functionality of the implemented grayscale conversion methods, a sample colored image was converted using both the sequential C and planar-float NASM AVX2 implementations. The resulting images were visually inspected to verify that the original colors were converted into grayscale while preserving recognizable image details. Additionally, a byte-for-byte comparison using the Windows *fc /b* command was performed to confirm that both implementations produced identical BMP output files for the selected test image.

V. Build and Measurement Environment

The project was developed, compiled, and tested using a Windows x64 environment with Microsoft Visual Studio and NASM.

Component	Configuration
Processor	AMD Ryzen 7 8845HS with Radeon 780M Graphics
Operating System	Windows 11 Home Single Language
OS Build	26200.9974
System-Reported Physical Memory	13.75 GB
Development Environment	Visual Studio 2022 Community
C Compiler	MSVC 19.44.35229 (x64)
Assembler	NASM 3.02
Build Configuration	Optimized x64 (/O2 /fp:precise)
Vectorization Diagnostics	/Qvec-report:2
Active Power Plan	Balanced

VI. Benchmark Methodology

Two measurement categories were used to distinguish the computational performance of the SIMD kernel from the overall grayscale conversion performance.

Primary: Complete in-memory conversion

Measures conversion from packed 24-bit BMP pixels already in memory to a complete grayscale BGR output. Includes output copying, channel extraction, scratch-memory allocation, grayscale computation, and output preparation as applicable.

Secondary: Kernel-only performance

Measures grayscale computation using prepared floating-point channel arrays, including AVX2 tail handling. Excludes channel extraction, BGR repacking, and memory allocation.

Benchmark protocol

The following procedure was applied to each configuration:

1. Five warm-up executions were performed before timing.
2. Thirty measured executions were recorded, with the execution order rotated.
3. Mean and median execution times were calculated.
4. Speedup was calculated relative to the appropriate scalar implementation.

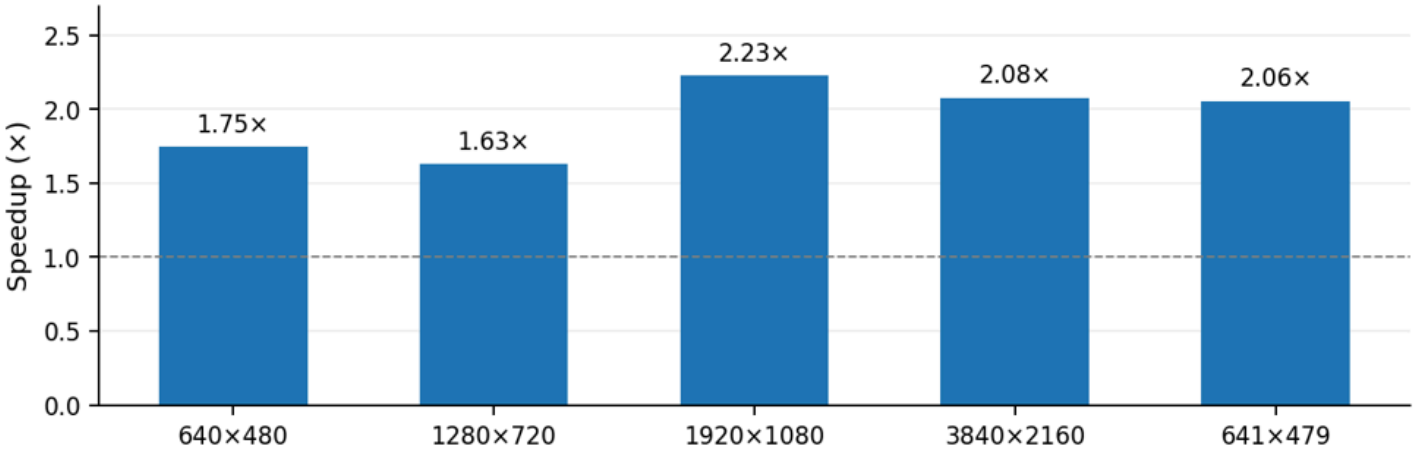
Speedup was determined using:

$$\text{Speedup} = T_{\text{scalar}} / T_{\text{SIMD}}$$

For complete in-memory conversion, sequential C was used as the baseline. For kernel-only performance, the prepared-float scalar C implementation served as the baseline. These measurements were analyzed separately because kernel-only speedup does not necessarily represent application-level performance improvement.

VII. Primary Results: Complete In-memory Conversion

Resolution	Seq. C (ms)	Planar C (ms)	Planar AVX2 (ms)	AVX2 speedup
640 × 480	5.06	6.96	2.90	1.75×
1280 × 720	15.66	21.97	9.61	1.63×
1920 × 1080	34.24	42.29	15.36	2.23×
3840 × 2160	138.28	172.12	66.50	2.08×
641 × 479	4.93	6.38	2.40	2.06×



The planar NASM AVX2 path had lower mean complete-conversion time than sequential C for all five image sizes, with speedups from 1.63× to 2.23×. Planar C was slower than sequential C in all five tests, indicating that channel separation without SIMD did not pay for its preprocessing cost in these measurements. These observations do not yet establish how a complete direct packed-BMP AVX2 method will perform.

VIII. Secondary Results: Prepared-float Kernel-only diagnostic

Resolution	Prepared scalar C (ms)	AVX2 + tail (ms)	Speedup
640 × 480	4.292	0.110	39.07×
1280 × 720	12.689	0.742	17.09×
1920 × 1080	28.167	1.957	14.39×
3840 × 2160	112.413	9.753	11.53×
641 × 479	4.038	0.108	37.39×

The prepared-float kernel-only speedups ranged from 11.53× to 39.07×, considerably larger than the full-conversion speedups because image extraction, output preparation, and allocations were excluded. Their magnitudes should not be attributed solely to the eight-lane width of AVX2: scalar rounding and packing work, generated instructions, memory behavior, and timing variation also require examination.

IX. Temporary-memory Requirements

Let

$$N = width * absolute\ image\ height$$

The three planar single-precision channel arrays use 12N bytes of image-sized scratch storage. The current planar NASM AVX2 path additionally creates an N-byte grayscale staging buffer, totaling 13N bytes; sequential C requires no additional image-sized scratch array. The figures exclude common BMP input/output buffers, allocator overhead, and benchmark-only arrays.

Resolution	Sequential C (MB)	Planar C (MB)	Planar AVX2 (MB)
640 × 480	0	3.69	3.99
1280 × 720	0	11.06	11.98
1920 × 1080	0	24.88	26.96
3840 × 2160	0	99.53	107.83
641 × 479	0	3.68	3.99

The packed-BGR extraction prototype uses bounded per-group working storage, but the completed packed-to-grayscale path and its additional memory costs have not yet been measured.

X. Measurement Limitations

Although the preliminary benchmarking results demonstrate performance improvements from the planar NASM AVX2 implementation, several limitations must be considered when interpreting the measurements.

Single benchmark session

The reported results were obtained from one Windows benchmarking session rather than multiple controlled sessions. Background processes, operating-system activity, and hardware conditions may influence the measured execution times.

Observed timing variation

Individual measurements showed noticeable fluctuations. For the 3840 × 2160 planar NASM AVX2 complete conversion, execution times ranged from approximately 54.85 ms to 93.70 ms, with a median of 65.18 ms. Such variation can influence the calculated mean and resulting speedup.

Required final validation.

For the final milestone, the group will repeat the measurements under documented power and workload conditions, analyze timing variability, inspect the generated scalar machine code, and evaluate the completed direct packed-BMP AVX2 implementation using equivalent input and output boundaries.

XI. Next work through Week 13

Task	Expected evidence
Complete packed-BMP grayscale path	Direct BGR extraction, identical grayscale arithmetic, safe output writes
Expand correctness and safe-load checks	Exact-byte output comparisons, odd widths, exact-sized/protected buffers
Benchmark all completed methods fairly	Same packed-input → packed-output boundaries; 30+ measurements per configuration
Quantify memory and improve reproducibility	Scratch allocations, timing dispersion, CPU and compiler configurations

XII. AI-use Disclosure and Group Authorship

Artificial intelligence (AI) tools, particularly ChatGPT, were utilized during the development of the project to assist with programming, debugging, testing, performance evaluation, and documentation. It assisted in interpreting execution times, speedup measurements, and temporary-memory requirements, along with organizing the project documentation and providing a template for the mid-project report.

The group compiled and tested the implementations using Microsoft Visual Studio 2022 and NASM 3.02 in a Windows x64 environment. The project successfully passed 226 automated correctness test cases, including tests for BMP handling, grayscale conversion, and SIMD channel extraction. Additional benchmark-framework and memory-boundary tests were also completed. The group further verified that the sequential C, planar-float C, and planar-float NASM AVX2 implementations produced identical BMP outputs for the tested image. Responsibility for verifying the correctness, understanding the implementation, and ensuring the integrity of the submitted work remains with the group members.

The current NASM AVX2 implementations were developed and have been successfully compiled and tested by the group. These implementations serve as preliminary prototypes for the mid-project milestone.

XIII. References

[1] OpenCV, “Color Conversions,”
https://docs.opencv.org/4.13.0/de/d25/imgproc_color_conversions.html
[2] Simd Library, “Simd Image Processing Library,”
https://github.com/ermig1979/Simd
[3] Intel Corporation, Intel 64 and IA-32 Architectures Software Developer’s Manuals,
https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
[4] Microsoft Learn, BITMAPINFOHEADER structure,
https://learn.microsoft.com/en-us/windows/win32/api/wingdi/ns-wingdi-bitmapinfoheader
[5] Group-collected Windows x64 terminal output and latest benchmark_results_new.csv and benchmark_results_new_samples.csv (November 2026).